

Load Test Report: Test 1 of 3, 30 Concurrent Users

Date: 26 Sep 2026, 10:05 – 10:11 IST
Target: `https://api.iverto.ai/devhoste1`, Hostel **v2** API (`/v2/tenants/acme/...`)
Tool: k6 v2.2.0, `k6/v2/users.js`, run from one Windows laptop (one public IP)
Traffic: reads only. No passes, decisions or other writes were sent.

1. Verdict

PASS on the overall SLOs, FAIL for staff screens.

SLO	Target	Result	
HTTP error rate	< 1%	0.36% (29 / 8,029)	✓
Checks passed	> 99%	99.63%	✓
Latency p95	< 800 ms	636 ms	✓
Latency p99	< 2 s	1.26 s	✓
GET /dashboard success	100%	27% (27 of 37 calls returned 500)	X
GET /roster success	100%	0% (1 × 500 , 1 × 30 s timeout)	X

Students and guardians had **zero errors**. **All 29 failures came from warden/admin screens.**

2. Test setup

Item	Value
Model	Closed: each virtual user opens the app (one role journey of 6–15 requests), then idles 3–8 s, then repeats
Profile	0 → 30 users over 1 min · hold 30 users for 4 min · ramp down 30 s (total 5 m 37 s)
Role mix	student 80% · guardian 15% · warden 4% · admin 1%
Token pool	35 tokens (24 students, 6 guardians, 3 wardens, 2 admins), logged in 10:01–10:05, 0 × 429
Request timeout	30 s

3. Results

3.1 Totals

Metric	Value
Requests	8,029 (23.8 req/s average, ~ 28–30 req/s at steady state)
Journeys (app opens)	892 (2.65 per second)
Latency p50 / p90 / p95 / p99 / max	118 ms / 495 ms / 636 ms / 1.26 s / 30 s
Journey duration p50 / p95	9.2 s / 14.1 s (includes think time)
Failed	29 = 28 × 500 + 1 × timeout

3.2 Timeline (15 s buckets)

Time	Users	req/s	p50	p95	Errors
0–60 s	ramp 0 → 30	6 → 28	~100 ms	260–380 ms	0%
60–75 s	30	32	96 ms	466 ms	0%
75–120 s	30	22–30	110–280 ms	600–840 ms	0.5–0.9% (500)
120–195 s	30	28–30	95–115 ms	390–640 ms	0–0.5%
195–225 s	30	21–25	220–380 ms	870 ms – 1.23 s	0.5–1.0%
225–315 s	30	24–30	100–240 ms	510–820 ms	0–0.8%
315 s → end	ramp down	9	100 ms	250 ms	0%

Throughput held at about **28–30 req/s**, the same ceiling found on 24 Sep. Every dip in req/s (75 s, 195–210 s) lines up with slow `/dashboard` calls. While those run, the whole server slows down: p50 more than doubles.

3.3 Per endpoint (sorted by p95)

Endpoint	Calls	p50	p95	Max	Errors
GET <code>/roster</code>	2	30 s	30 s	30 s	2 (100%) : 1 × 500, 1 × timeout
GET <code>/dashboard</code>	37	12.8 s	19.3 s	19.7 s	27 × 500 (73%)
GET <code>/groups</code>	5	5.8 s	9.6 s	9.6 s	0
GET <code>/memberships</code>	1	2.5 s	2.5 s	2.5 s	0
GET <code>/roles</code>	1	2.2 s	2.2 s	2.2 s	0
GET <code>/me/children/:id</code>	30	709 ms	1.42 s	1.98 s	0
GET <code>/me/children</code>	123	707 ms	1.37 s	1.73 s	0
GET <code>/me/student</code>	732	405 ms	818 ms	2.22 s	0
GET <code>/me/curfew</code>	732	415 ms	751 ms	3.99 s	0
GET <code>/me/branding</code>	892	197 ms	483 ms	1.31 s	0
GET <code>/app-config</code>	892	95 ms	408 ms	777 ms	0
GET <code>/sites</code>	769	171 ms	407 ms	990 ms	0
GET <code>/me/permissions</code>	1,081	90 ms	342 ms	3.18 s	0
GET <code>/me/permissions/summary</code>	732	87 ms	415 ms	3.39 s	0
GET <code>/me/notifications/unread-count</code>	892	89 ms	308 ms	724 ms	0
All other endpoints	590	≤ 133 ms	≤ 450 ms		0

4. What went wrong

- `/dashboard` **fails 73% of the time with only 3 wardens and 2 admins in the pool**. Successful calls take 10–20 s. Failures are 500 INTERNAL_ERROR at 12–20 s, which matches a server-side statement or pool-acquire timeout, not a crash.
- `/roster` **never succeeded**: one 500, and one call hit the 30 s client timeout.

3. **Staff reports slow down everyone.** Each time `/dashboard` runs, student p50 goes from ~100 ms to 220–380 ms and throughput drops by up to 30%. The heavy queries hold DB connections that student requests are waiting on.
4. **30 users already sit on the ~30 req/s ceiling.** 30 users making one app-open every ~11 s produce ~28–30 req/s, the same knee measured on 24 Sep. There is **no headroom** above this.
5. **Guardian endpoints are slower than student ones:** `/me/children` and `/me/children/:id` p95 $\approx$ 1.4 s at this low load. This suggests per-child N+1 lookups.
6. Test data: `parent.acm0001@example.com` still fails login (shared email between two guardians, 401 with both passwords).

5. Fixes

#	Problem	Fix
1	<code>/dashboard</code> 12–20 s, 73% 500	Replace per-request computation with one <code>GROUP BY</code> aggregate over tenant/site. Cache the result 15–30 s (Redis or materialized view). Target < 100 ms. Run <code>EXPLAIN (ANALYZE, BUFFERS)</code> on it first.
2	<code>/roster</code> 30 s / 500	Paginate in SQL (keyset on an indexed column), not in memory. Add <code>(tenant_id, site_id)</code> and <code>(tenant_id, status)</code> indexes. Target < 300 ms for <code>limit=20</code> .
3	Staff queries starve students	Set <code>statement_timeout</code> $\approx$ 5 s. Give reports a separate small pool (or a read replica) so they cannot use up the connections student traffic needs.
4	<code>/groups</code> 6–10 s, <code>/roles</code> / <code>/memberships</code> 2+ s	Remove N+1 (group $\rightarrow$ members), add a <code>(group_id)</code> index on members.
5	Guardian <code>/me/children*</code> 1.4 s p95	Load children and their guardians in one join or <code>IN (...)</code> query. Index <code>(student_id)</code> / <code>(guardian_id)</code> on the link table.
6	<code>parent.acm0001</code> login	Give each guardian a unique email. Add a unique constraint on principal email per tenant.

6. Raw files

`load-test/results/2026-09-26/users-30.{csv,json,log}`, `users-30.csv.analysis.json`, `pool-30.log`.
 Reproduce: `node scripts/login-pool-v2.mjs && k6 run -e USERS=30 -e HOLD=4m k6/v2/users.js`